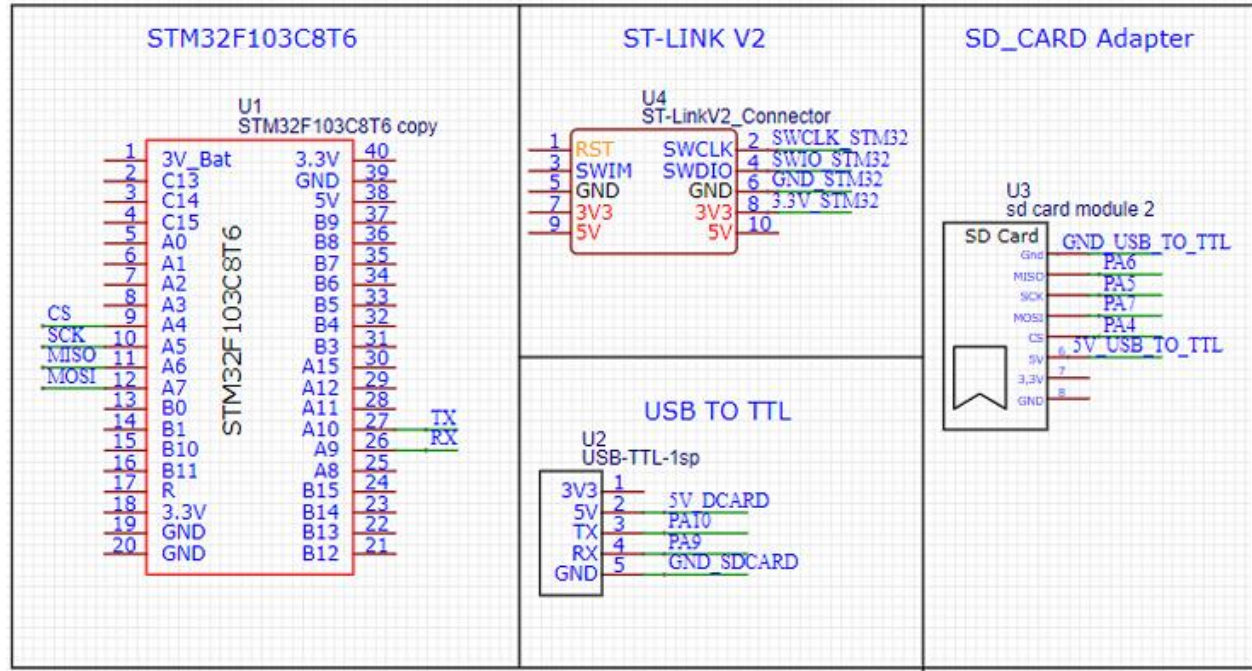


4 SPI(SD Card, đọc/ghi dạng FATFS)



4.1 Định nghĩa

FatFs là thư viện mã nguồn mở chuyên dụng cho hệ thống nhúng, được viết bằng ANSI C nên hoàn toàn độc lập với phần cứng và tách biệt với lớp I/O đĩa cấp thấp. Đóng vai trò là lớp phần mềm trung gian (Middleware)

FatFs cung cấp các API tiêu chuẩn giúp người dùng quản lý dữ liệu dưới dạng tệp tin (như .txt, .csv) thay vì phải xử lý các sector vật lý phức tạp. Nhờ hỗ trợ đầy đủ các định dạng FAT, dữ liệu từ hệ thống nhúng luôn đảm bảo tính tương thích cao và có thể đọc/ghi trực tiếp trên các hệ điều hành máy tính như Windows, Linux hay macOS

4.2 ĐẶC ĐIỂM

- Tối ưu hóa tài nguyên hệ thống

- + Cơ chế truyền dẫn không cần CPU can thiệp trực tiếp giúp vi điều khiển rảnh tay xử lý các tác vụ chính.

- + Dấu chân RAM cực nhỏ từ 2 đến 10 kB giúp vận hành ổn định ngay cả trên các dòng STM32 tài nguyên thấp.

- + Cấu trúc mã nguồn ANSI C tách biệt với tầng vật lý mang lại khả năng độc lập phần cứng và tùy biến linh hoạt.

- Hiệu suất truyền tải dữ liệu

- + Loại bỏ độ trễ phần mềm giữa các tầng kiến trúc giúp duy trì tốc độ bus SPI ở mức tối đa.

- + Hỗ trợ Burst Transfer truyền dữ liệu liên tục giúp ghi nhật ký cảm biến vào file .csv ổn định và nhanh chóng.

- + Khả năng cấu hình kích thước sector lên đến 4 KB giúp tối ưu hóa hiệu suất dựa trên đặc tính từng loại thẻ.

- Độ tin cậy và an toàn dữ liệu

- + Cơ chế _FS_LOCK kiểm soát truy cập chặt chẽ nhằm ngăn ngừa rủi ro hỏng cấu trúc FAT khi ghi trùng lặp.

- + Đảm bảo tính tái nhập và an toàn đa luồng khi tích hợp RTOS giúp xử lý song song nhiều tệp tin đồng thời.

- + Tự động cập nhật nhãn thời gian RTC và kích thước tệp sau mỗi phiên ghi giúp duy trì tính toàn vẹn thông tin.

- Tính tương thích và mở rộng

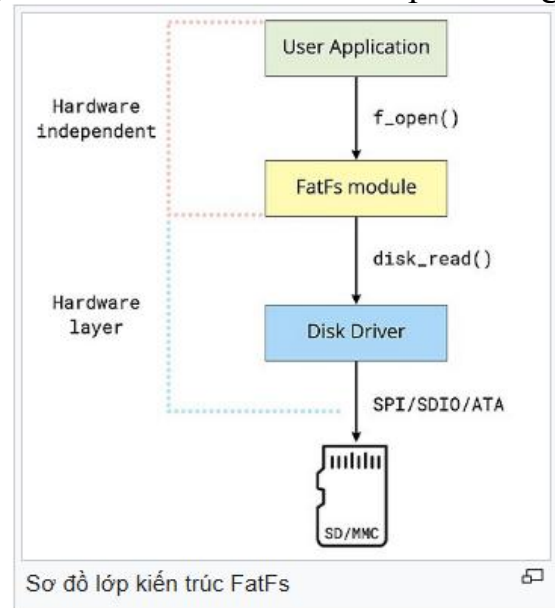
- + Định dạng FAT/exFAT tương thích hoàn toàn với Windows giúp trao đổi file .txt và .csv với máy tính dễ dàng.

- + Hỗ trợ tên tệp dài LFN lên đến 255 ký tự giúp tổ chức dữ liệu trực quan thay vì giới hạn 8.3 truyền thống.

- + Quản lý đồng thời tới 10 phân vùng vật lý khác nhau mang lại khả năng mở rộng không gian lưu trữ lớn

4.3 Nguyên lý

Nguyên lý vận hành cốt lõi của hệ thống dựa trên mô hình kiến trúc phân tầng nghiêm ngặt, giúp tách biệt hoàn toàn giữa logic xử lý tệp tin cấp cao và việc điều khiển phần cứng vật lý cấp thấp. Toàn bộ lộ trình dữ liệu bắt đầu từ tầng Ứng dụng, nơi người lập trình sử dụng các hàm API tiêu chuẩn như `f_open`, `f_read`, `f_write` để thực hiện thao tác trên các tệp .txt hoặc .csv. Khi một lệnh ghi được thực thi, mô-đun FatFs Middleware sẽ tiếp nhận yêu cầu và thực hiện các thuật toán tính toán logic nhằm xác định vị trí các cụm dữ liệu và phân vùng cung từ tương ứng trên bảng phân phối tệp. Do FatFs được thiết kế tuân thủ ANSI C và tách biệt hoàn toàn với tầng I/O, nó đóng vai trò điều phối mà không phụ thuộc vào bất kỳ đặc tính phần cứng cụ thể nào của vi điều khiển.

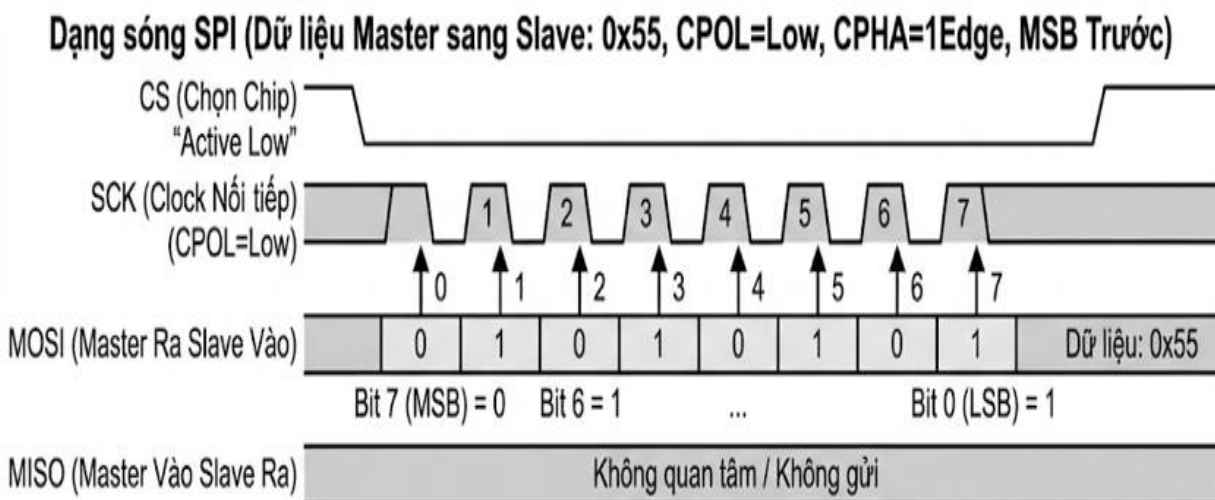


Tại tầng giao tiếp vật lý, yêu cầu từ FatFs được chuyển đổi thành các khung truyền dữ liệu thông qua lớp Low-level Disk I/O. Đây là cầu nối quan trọng đòi hỏi người triển khai phải cung cấp các hàm điều khiển ngoại vi SPI cụ thể cho STM32, bao gồm tối thiểu ba giao diện bắt buộc: `disk_status` để kiểm soát trạng thái thiết bị, `disk_initialize` để thực hiện các thủ tục chào hỏi với thẻ nhớ, và `disk_read/write` để thực hiện việc dịch bit dữ liệu trên đường truyền. Quá trình này diễn ra theo cơ chế truy xuất khối, trong đó dữ liệu từ bộ đệm của vi điều khiển được đẩy qua bus SPI xuống các cung từ có kích thước 512 bytes trên thẻ SD. Song song với đó, mô-đun RTC sẽ cung cấp thông tin thời gian thực để FatFs tự động cập nhật nhãn thời gian vào thư mục gốc, giúp tệp tin được lưu trữ một cách khoa học và chính xác.

Đối với các hệ thống yêu cầu độ phức tạp cao như quản lý tên tệp dài hoặc hoạt động trong môi trường đa nhiệm, nguyên lý vận hành còn bao gồm các cơ chế kiểm soát an toàn dữ liệu nghiêm ngặt. Khi tính năng LFN được kích hoạt, hệ thống sẽ sử dụng các hàm chuyển đổi mã Unicode như `ff_convert` và cấp phát một bộ đệm làm việc riêng trên ngăn xếp hoặc vùng nhớ heap để xử lý các tên tệp lên tới 255 ký tự một cách minh bạch đối với ứng dụng. Để đảm bảo tính toàn vẹn của cấu trúc FAT, cơ

chế khóa tệp `_FS_LOCK` sẽ kiểm soát các hành vi truy cập trùng lặp, ngăn chặn việc đổi tên hoặc xóa tệp tin trong khi đang thực hiện tiến trình ghi dữ liệu. Đặc biệt, trong môi trường RTOS, các thao tác tệp tin trên cùng một phân vùng sẽ được đồng bộ hóa thông qua các đối tượng của hệ điều hành, cho phép tạm dừng các tác vụ tranh chấp cho đến khi tiến trình trước đó hoàn tất hoặc đạt ngưỡng thời gian chờ quy định

4.4 Cấu trúc gói tin và dạng sóng



Hình 1: Dạng sóng SPI truyền 0x55 với cấu hình được chỉ định.

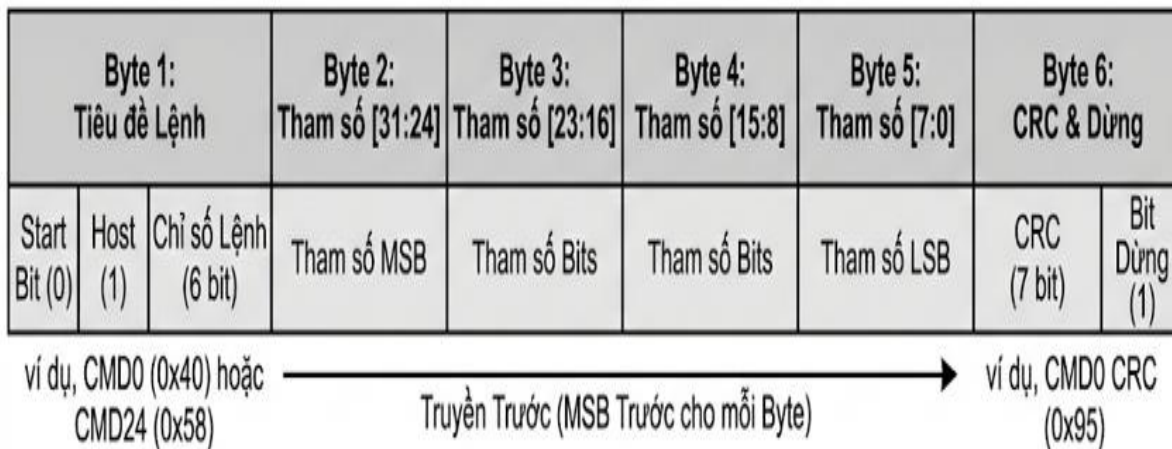
Đoạn 1: Lớp vật lý

Hình 1, dạng sóng của giao tiếp SPI, đây chính là 'lớp vật lý' – nền móng thấp nhất để vi điều khiển có thể chạm tới thẻ nhớ

Sơ đồ dạng sóng minh họa quá trình truyền tải byte dữ liệu 0x55 từ Master sang Slave. Quá trình này bắt đầu khi tín hiệu chọn chip CS (Chip Select) được kéo xuống mức thấp. Xung nhịp đồng bộ SCK được thiết lập ở chế độ CPOL=Low (trạng thái nghỉ là mức 0) và lấy mẫu dữ liệu tại cạnh lên đầu tiên (CPHA=1st Edge). Dữ liệu được truyền trên đường MOSI theo quy tắc bit có trọng số lớn nhất đi trước (MSB First), thể hiện rõ qua chuỗi bit nhị phân 01010101. Trong khi đó,

đường MISO ở trạng thái không xác định do Master đang giữ vai trò

Cấu trúc Gói Lệnh Thẻ SD (Chế độ SPI, 48 bit)



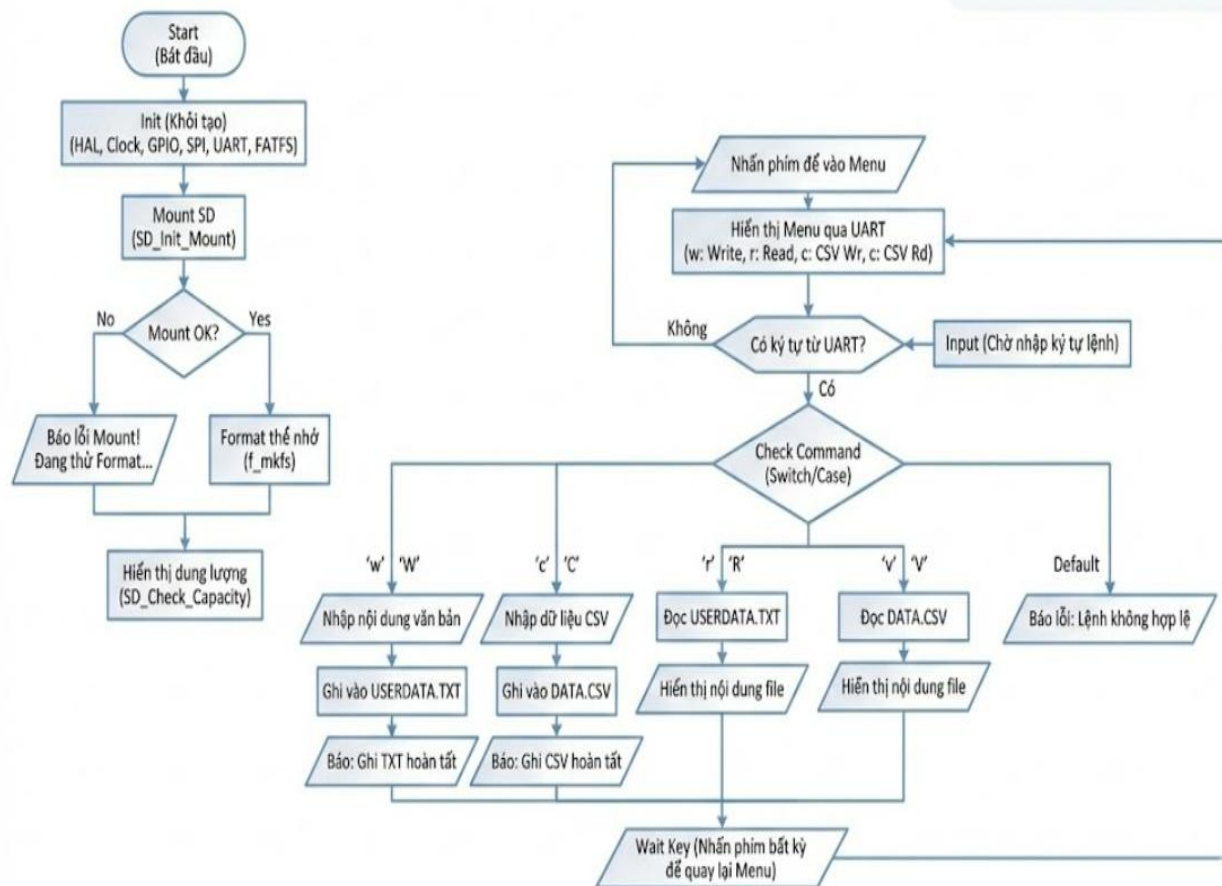
Hình 2: Gói Lệnh Thẻ SD chuẩn 6-byte.

Đoạn 2: Môi liên hệ với hệ thống tập tin FATFS

Khi triển khai việc lưu trữ dữ liệu thực tế, lớp giao thức SPI và các gói lệnh này chính là nền tảng để vận hành thư viện FATFS

Cụ thể, khi người lập trình gọi các hàm cao cấp như `f_open` để mở file hay `f_write` để ghi dữ liệu, thư viện FATFS sẽ tính toán vị trí các sector trên thẻ nhớ và chuyển đổi chúng thành các mã lệnh tương ứng như CMD17 (đọc) hoặc CMD24 (ghi). Sau đó, các lệnh này lại được băm nhỏ thành dạng sóng SPI vật lý để truyền xuống phần cứng. Nói cách khác, nếu không có cấu trúc gói tin và dạng sóng SPI chuẩn xác như trong hình, hệ thống FATFS sẽ không thể quản lý được các tập tin, thư mục hay phân vùng dữ liệu trên thẻ nhớ SD

4.5 Lưu đồ trình tự



Lưu đồ trình tự hoạt động

Quy trình hoạt động :

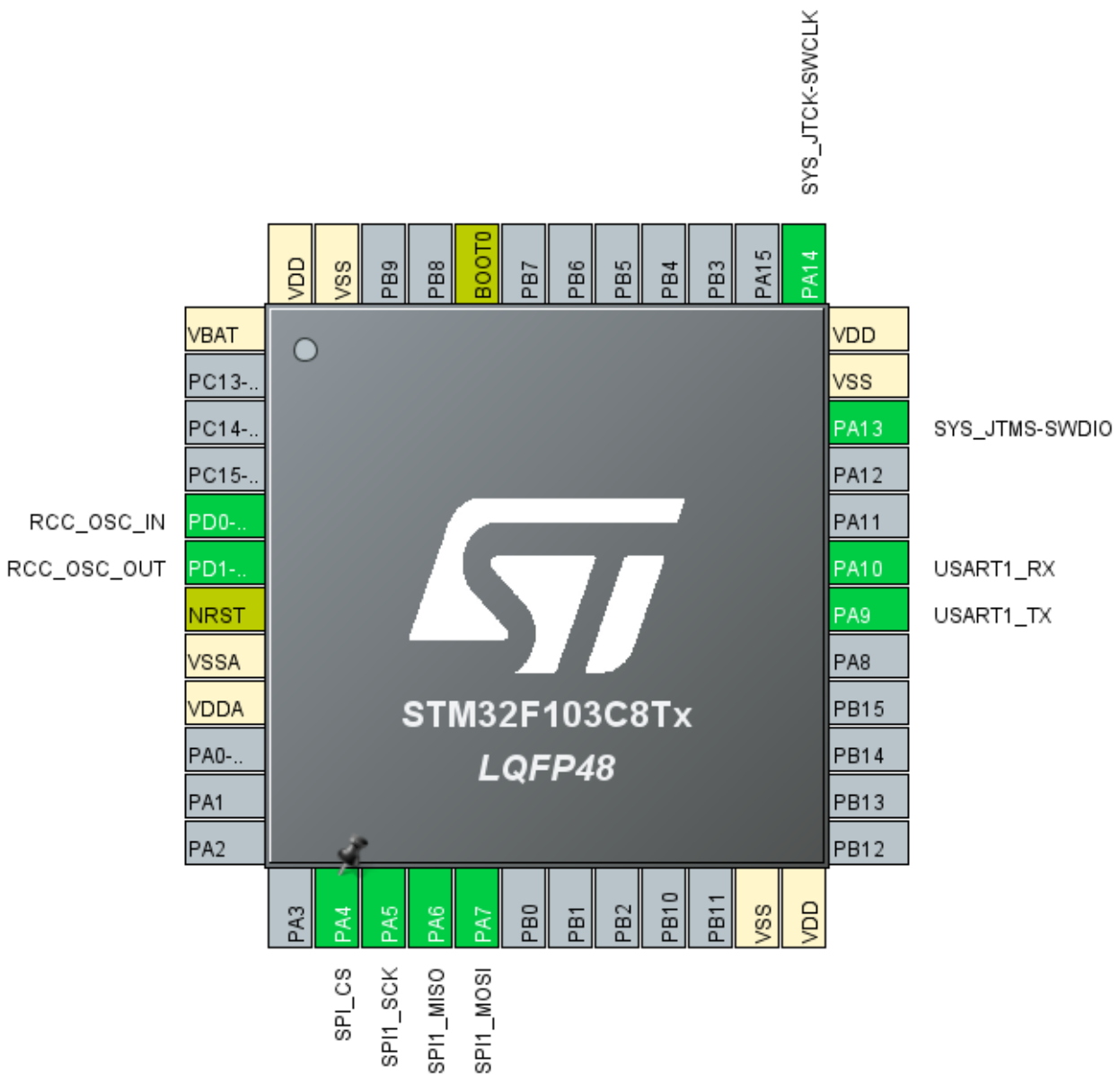
Khởi tạo: Đầu tiên, máy sẽ thiết lập các giao thức (SPI, UART) và thử kết nối với thẻ nhớ. Nếu thẻ lỗi, máy tự động định dạng (format) lại, sau đó thông báo dung lượng thẻ để xác nhận đã sẵn sàng.

Chọn lệnh: Một menu sẽ hiện lên màn hình máy tính qua cổng UART. Bạn nhập các phím tương ứng: w/r để ghi hoặc đọc file văn bản (.txt), và c/v để ghi hoặc đọc file dữ liệu (.csv).

Thực hiện và Lặp lại: Tùy vào phím bạn chọn, hệ thống sẽ yêu cầu nhập dữ liệu để lưu vào thẻ hoặc đọc dữ liệu từ thẻ hiện lên màn hình. Xong việc, bạn chỉ cần nhấn phím bất kỳ là máy quay lại menu ban đầu để tiếp tục lệnh khác.

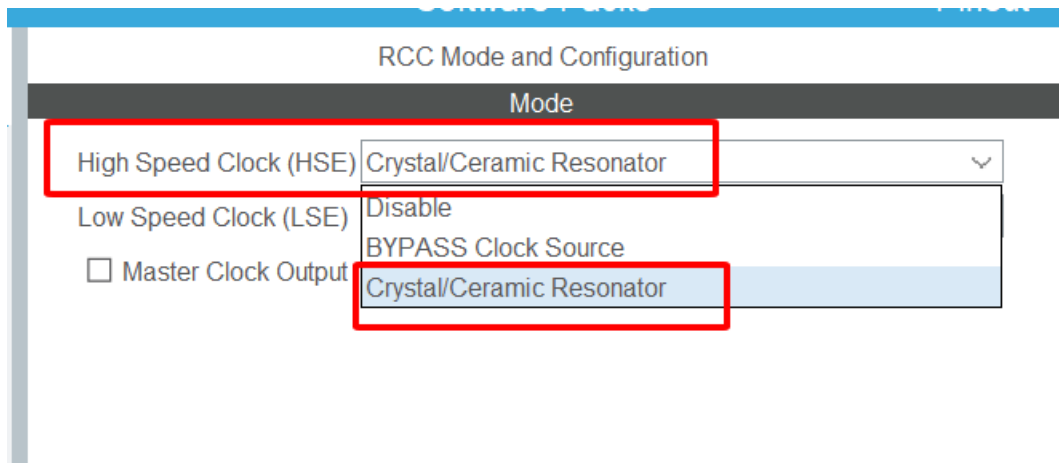
4.6 Chi tiết quy trình

A. Giải thích trên STM32Cube



* Giải thích thiết lập chọn giao diện phần cứng phần cứng

+ Cấu hình System Clock (RCC)



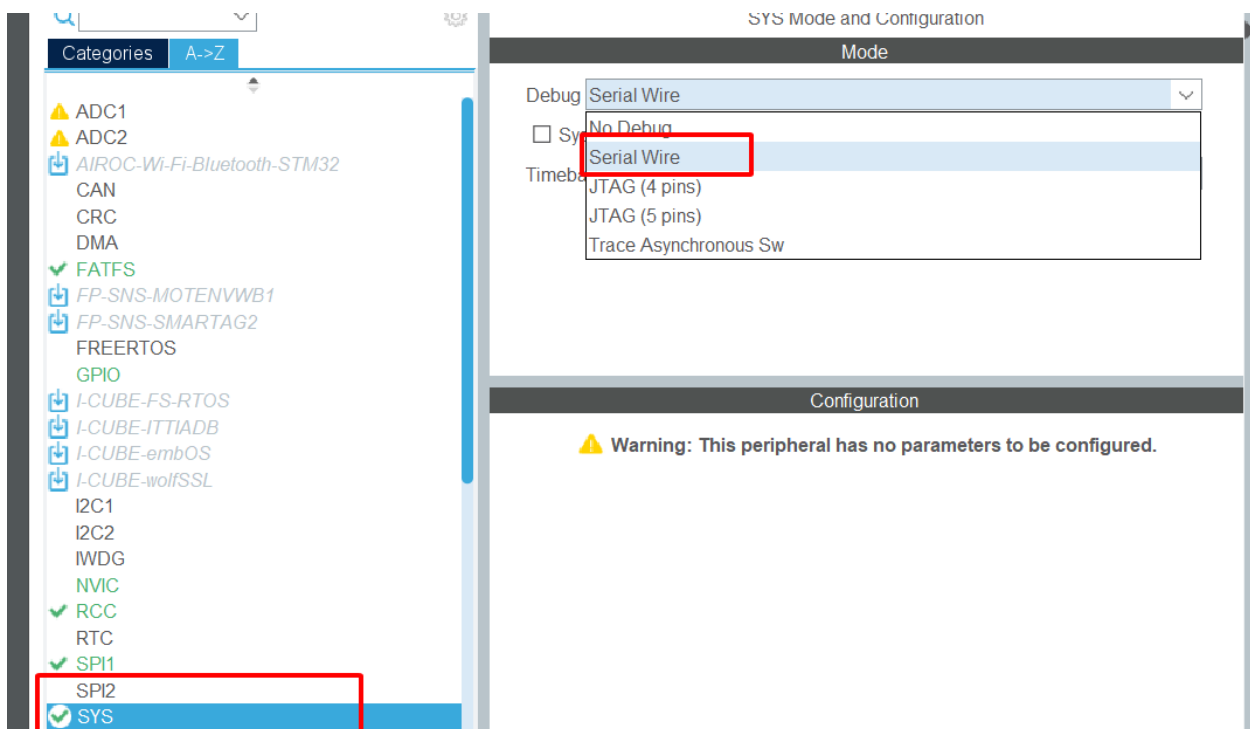
Chọn: High Speed Clock (HSE) -> Crystal/Ceramic Resonator

Sử dụng thẻ nhớ SD và giao tiếp UART tốc độ cao. Nên thạch anh nội (HSI) thường không đủ chính xác và ổn định bằng thạch anh ngoại (HSE)

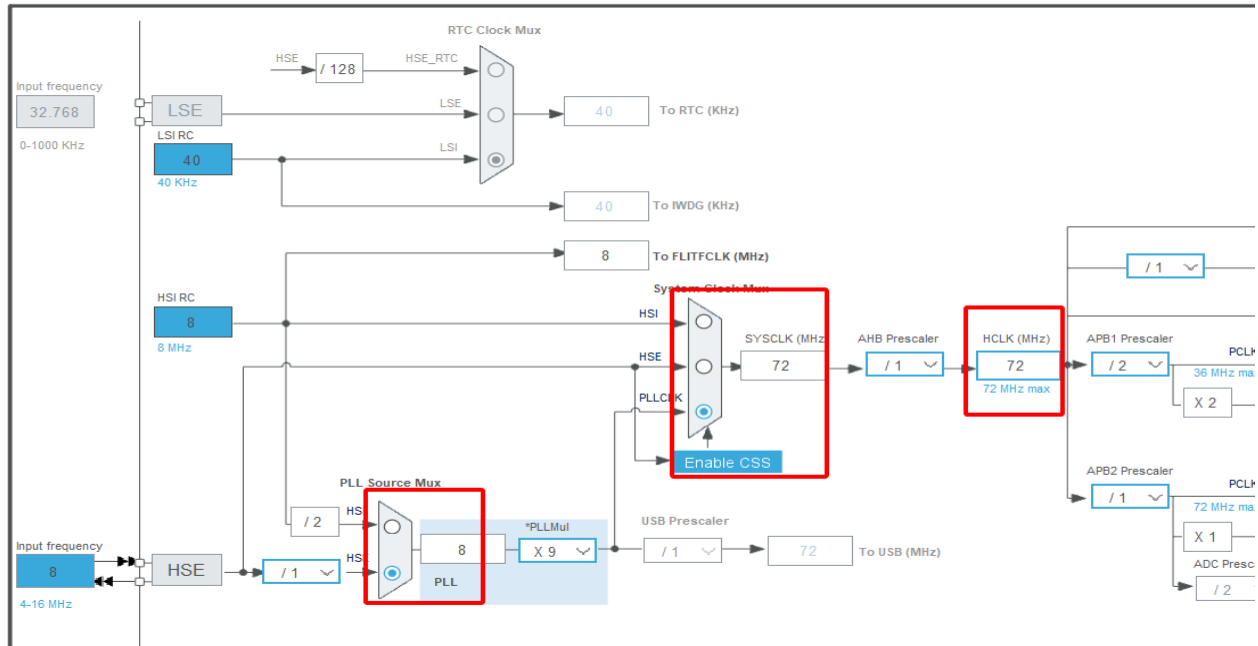
+ Cấu hình System Core (Hệ thống cốt lõi)

Chọn: Debug -> Serial Wire

Để nạp code và debug qua mạch ST-Link. Nếu không chọn cái này, sau khi nạp code lần đầu, chip có thể bị khóa và không nhận mạch nạp nữa



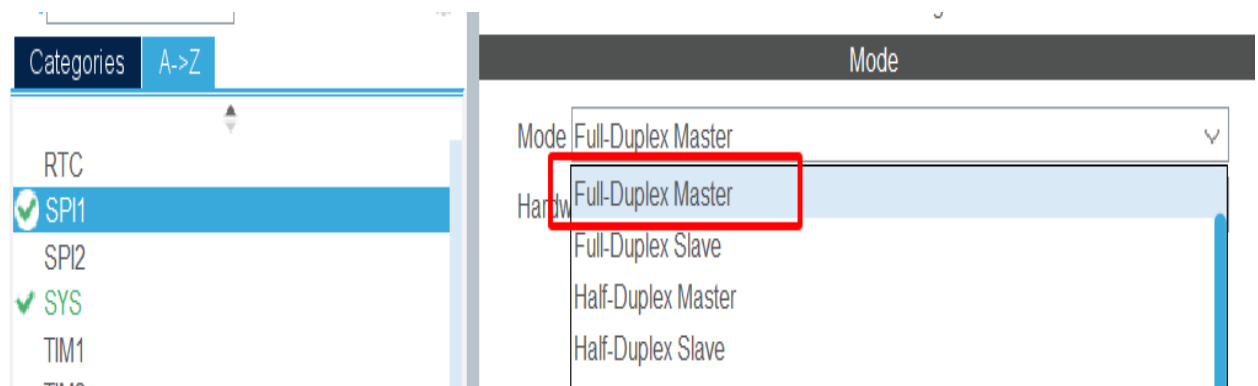
+ Cấu hình Clock Configuration

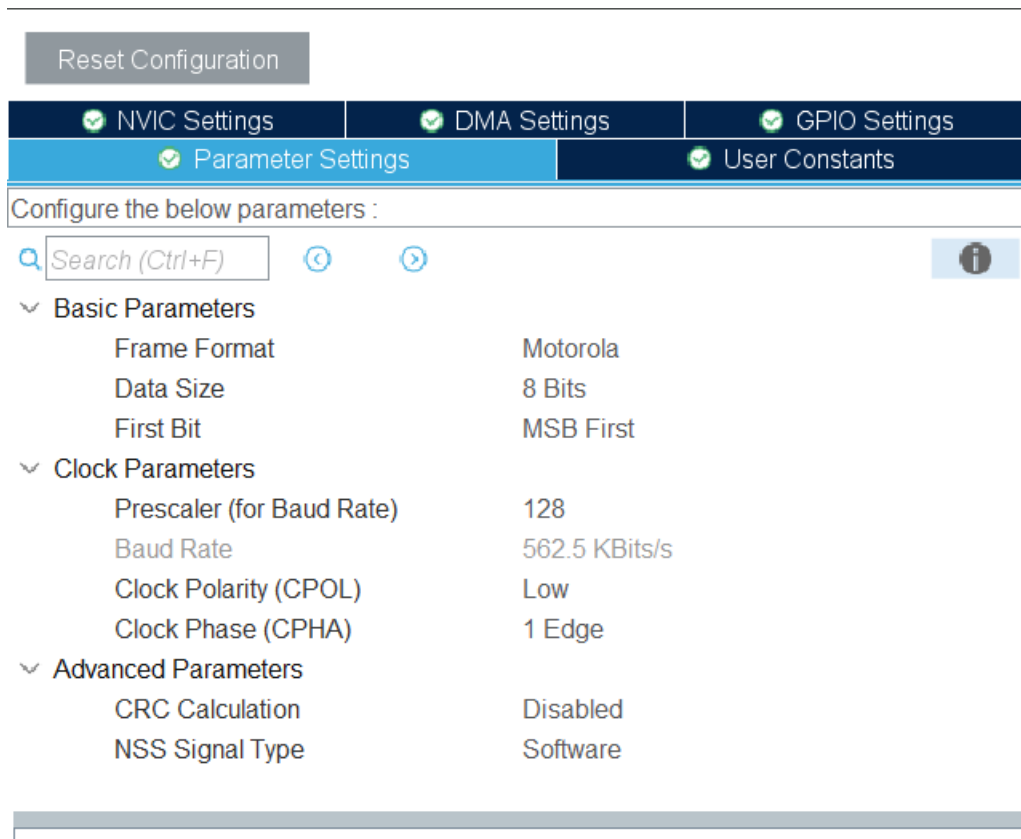


PLL Source: Chọn HSE , System Clock Mux: Chọn PLLCLK ,HCLK (MHz): Gõ 72 và nhấn Enter (Phần mềm sẽ tự tính toán các bộ chia còn lại)

Vì điều khiển chạy ở tốc độ tối đa (72MHz), giúp giao tiếp SPI và xử lý dữ liệu FatFs nhanh nhất có thể

+ Cấu hình Connectivity SPI1 (Giao tiếp thẻ nhớ)



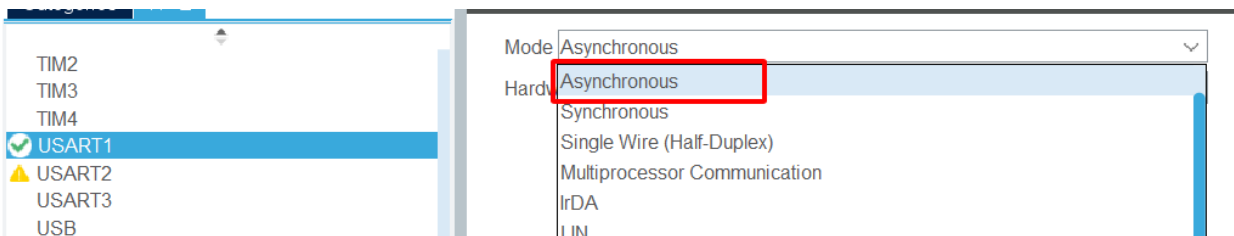


Mode: Chọn Full-Duplex Master để Vi điều khiển (Master) cần điều khiển thẻ nhớ (Slave) và cần cả gửi lệnh lẫn nhận dữ liệu về.

Hardware NSS Signal: Chọn Disable để điều khiển chân CS (Chip Select) thủ công bằng GPIO để linh hoạt hơn và tránh lỗi phần cứng của bộ SPI trên STM32F1

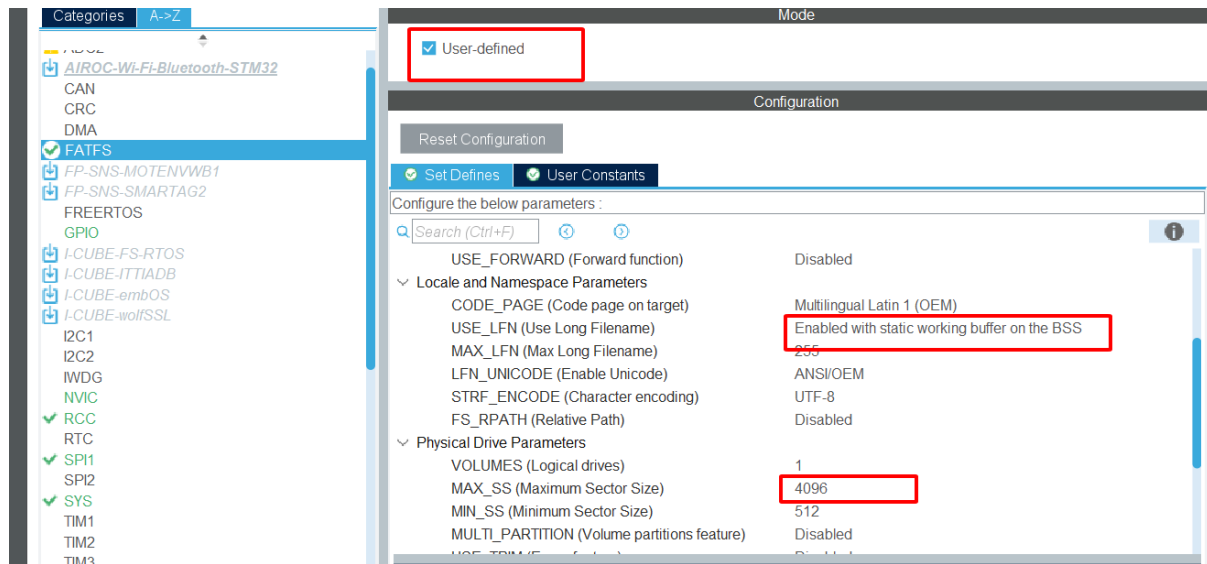
Parameter Settings : Data Size: 8 Bits (Thẻ nhớ giao tiếp theo byte), First Bit: MSB First (Chuẩn giao tiếp SD Card), Prescaler (Baud Rate Control): Chọn 128 hoặc 256 , CPOL: Low, CPHA: 1 Edge

+ Cấu hình USART1 (Giao tiếp máy tính)



Mode: Chọn Asynchronous (Bất đồng bộ) , Parameter Settings, Baud Rate: 115200 Bits/s, Word Length: 8 Bits, Parity: None, Stop Bits: 1

+ Cấu hình Middleware – FATFS

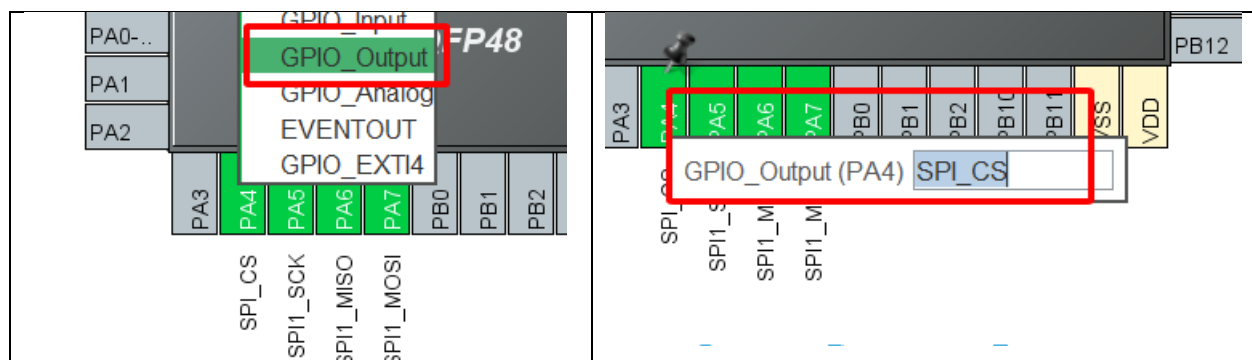


Vào mục Middleware -> FATFS Chọn: Tích vào ô User-defined

Chế độ "User-defined" cho phép bạn tự định nghĩa các hàm cấp thấp mà thư viện FatFs cần

Configuration (Tab bên cạnh): USE_LFN (Long Filename): Có thể chọn Enabled with static working buffer nếu muốn hỗ trợ tên file dài (trên 8 ký tự). MAX_SS chọn 4096 để hỗ trợ đa dạng các loại bộ nhớ có kích thước sector từ 512 byte đến 4096 byte

+ Cấu hình GPIO (Chân điều khiển) - Chân CS (Chip Select)

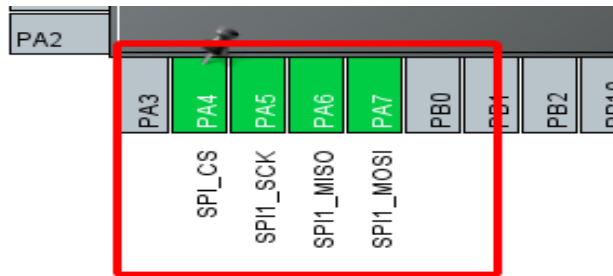


Chọn chân PA4, Click chuột trái, chọn GPIO_Output, Click chuột phải vào chân PA4, chọn Enter User Label, đặt tên là: SPI_CS . Là chân giao tiếp với thẻ nhớ SD

* Giải thích sơ đồ chân

- Giao tiếp ngoại vi : là các chân được dùng để kết nối với các module bên ngoài gồm:

Nhóm SPI1 (Giao tiếp nối tiếp tốc độ cao):



+ PA4 (SPI_CS): Chân chọn chip , dùng để kích hoạt thiết bị ngoại vi cụ thể khi có nhiều thiết bị trên cùng đường bus.

+ PA5 (SPI1_SCK): Chân tạo xung nhịp để đồng bộ hóa dữ liệu.

+ PA6 (SPI1_MISO): Nhận dữ liệu từ thiết bị ngoại vi truyền về vi điều khiển

+ PA7 (SPI1_MOSI): Truyền dữ liệu từ vi điều khiển ra ngoại vi

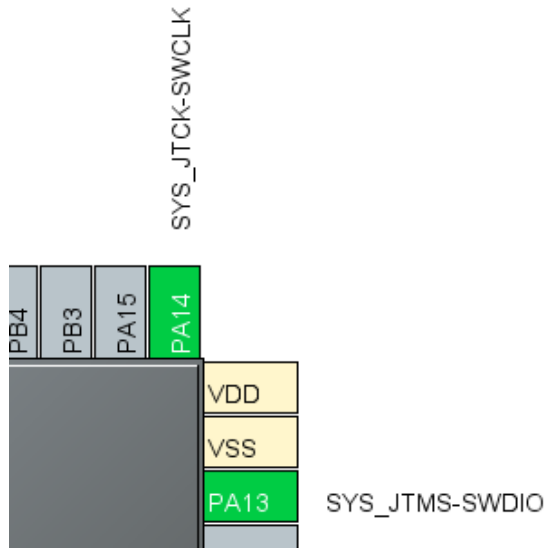
Nhóm USART1 (Giao tiếp nối tiếp UART):



+ PA9 (USART1_TX): Chân truyền dữ liệu đại diện cho giao thức Serial ,nối với chân RX của module khác hoặc mạch nạp.

+ PA10 (USART1_RX): Chân nhận dữ liệu , nối với chân TX của thiết bị khác.

Hệ thống và Nạp chương trình



+ PA13 và PA14 : Đây là hai chân của giao thức SWD Chúng được dùng để nạp code và debug thông qua các mạch nạp như ST-Link V2.

+ PD0 và PD1 : Hai chân này được nối với thạch anh ngoài (thường là 8MHz) để cung cấp xung nhịp chính xác cao cho vi điều khiển hoạt động thay vì dùng bộ dao động nội.

NRST: Chân Reset phần cứng. Khi kéo chân này xuống mức thấp (GND), vi điều khiển sẽ khởi động lại.

BOOT0: Chân quy định chế độ khởi động của chip (nạp qua UART hay chạy chương trình từ Flash)

B . Giải thích code

Tóm tắt luồng gọi thư viện:

```
main.c gọi -> sd_card.c (User Lib) -> gọi fatfs.h (Middleware) -> gọi fatfs_sd.c (Low-level Driver) -> gọi spi.c (HAL Driver) -> Điều khiển phần cứng.
```

- Giải thích code main.c

+ Phần khai báo thư viện

```

19  /* USER CODE END Header */
20  /* Includes -----
21  #include "main.h"
22  #include "fatfs.h"
23  #include "spi.h"
24  #include "usart.h"
25  #include "gpio.h"
26

```

Gọi các file header chứa định nghĩa phần cứng và cấu hình ngoại vi đã được tạo bởi STM32CubeMX (GPIO, SPI, UART, FatFs).

```

30  // Các thư viện C chuẩn để xử lý chuỗi (string.h) và in ấn (stdio.h)
31  #include <string.h>
32  #include <stdio.h>
33  #include <stdlib.h>

```

Gọi các thư viện chuẩn của ngôn ngữ C gồm string.h - để xử lý chuỗi (ví dụ: strlen, strcpy), stdio.h - để dùng hàm printf, sprintf, stdlib.h - để dùng các hàm chuyển đổi dữ liệu

```

#include "fatfs_sd.h"
#include "sd_card.h" // Thư viện cấp cao tự viết (chứa các hàm SD_Init_Mount, SD_Write_File...)

```

fatfs_sd.h: Driver cấp thấp kết nối thư viện FatFs với phần cứng SPI của STM32.

sd_card.h: Thư viện cấp cao do người dùng tự viết chứa các hàm tiện ích như SD_Init_Mount, SD_Write_File để code trong main gọn hơn

+ Phân định nghĩa biến toàn cục

```

77  #define BUFFER_SIZE 256 // Định nghĩa kích thước bộ đệm dữ liệu (256
78  char buffer[BUFFER_SIZE]; // Biến mảng lưu trữ tạm thời dữ liệu ngu
79

```

Tạo một hằng số BUFFER_SIZE là 256 Và khai báo mảng buffer chứa được 256 ký tự. Mảng này dùng để lưu dữ liệu tạm thời khi người dùng nhập từ bàn phím trước khi ghi vào thẻ nhớ

+Hàm main() - Khởi tạo hệ thống

```

HAL_Init(); //Khởi tạo thư viện HAL, cấu hình Flash prefetch và SysTick Timer (dùng cho các hàm HAL_Delay)

```

```

103  /* Configure the system clock */
104
105  SystemClock_Config(); //Cấu hình xung nhịp cho hệ thống, Hàm này được định nghĩa ở cuối file
106

```

```

110
111  /* Initialize all configured peripherals */ //khởi tạo các ngoại vi
112  MX_GPIO_Init(); //Cấu hình chân IO (chân CS của thẻ nhớ).
113  MX_SPI1_Init(); //Cấu hình giao tiếp SPI để nói chuyện với thẻ SD.
114  MX_USART1_UART_Init(); //Cấu hình giao tiếp SPI để nói chuyện với thẻ SD.
115  MX_FATFS_Init(); //Khởi tạo middleware hệ thống tệp tin FatFs.
116  /* USER CODE BEGIN 2 */
117

```

```

121  // --- KHOI TAO SD CARD TU THU VIEN MOI ---
122  // Mount (gắn) thẻ nhớ vào hệ thống. Nếu thẻ chưa định dạng, nó sẽ tự động Format// T
123  SD_Init_Mount();
124  // Kiểm tra và hiển thị dung lượng trống của thẻ nhớ lên màn hình.
125  SD_Check_Capacity();
126  // -----

```

+ Vòng lặp vô hạn while(1) - Bắt đầu vòng lặp chính. Khai báo biến cmd để lưu ký tự lệnh người dùng nhập

```

33  while (1)
34  {
35      /* USER CODE END WHILE */
36
37      /* USER CODE BEGIN 3 */
38      uint8_t cmd;
39

```

```

HAL_UART_Transmit(&huart1, (uint8_t*)"\\r\\n=== MENU SD CARD ===\\r\\n", 24, 100);
// ... (các dòng in hướng dẫn sử dụng w, r, c, v)
HAL_UART_Transmit(&huart1, (uint8_t*)"Nhập lệnh: ", 11, 100);

```

Gửi các chuỗi ký tự qua UART1 để hiển thị Menu lên màn hình máy tính (Serial Terminal)

```

if (HAL_UART_Receive(&huart1, &cmd, 1, HAL_MAX_DELAY) == HAL_OK) {

```

Hàm này chờ (do dùng HAL_MAX_DELAY) cho đến khi nhận được 1 ký tự từ máy tính và lưu vào biến cmd

```

if (cmd == '\\r' || cmd == '\\n') continue;

```


Nếu ký tự nhận được là phím Enter ,bỏ qua và quay lại đầu vòng lặp để tránh in lại menu không cần thiết

```
HAL_UART_Transmit(&huart1, &cmd, 1, 100);  
HAL_UART_Transmit(&huart1, (uint8_t*)"\r\n", 2, 100);
```

Gửi lại ký tự vừa nhận về máy tính để người dùng thấy mình vừa gõ gì, sau đó xuống dòng

```
if (cmd == 'w' || cmd == 'W') {  
    HAL_UART_Transmit(&huart1, (uint8_t*)"Nhập nội dung cần lưu: ", 23, 100);  
    UART_RxString(buffer, BUFFER_SIZE);  
    SD_Write_File("USERDATA.TXT", buffer);  
}
```

Lệnh 'w': Ghi file TXT Yêu cầu người dùng nhập nội dung

UART_RxString- Hàm tự viết (trong usart.c) để nhận cả một chuỗi ký tự dài từ bàn phím và lưu vào buffer SD_Write_File -Mở file USERDATA.TXT, ghi nội dung trong buffer vào cuối file, sau đó đóng file

```
else if (cmd == 'r' || cmd == 'R') {  
    SD_Read_File("USERDATA.TXT");  
}
```

Lệnh 'r': Đọc file TXT và Gọi hàm SD_Read_File để mở file USERDATA.TXT, đọc toàn bộ nội dung và gửi qua UART để hiển thị lên màn hình

```
else if (cmd == 'c' || cmd == 'C') {  
    HAL_UART_Transmit(&huart1, (uint8_t*)"Nhập dữ liệu CSV (VD: 10,20,30): ", 33, 100);  
    UART_RxString(buffer, BUFFER_SIZE);  
    SD_Write_File("DATA.CSV", buffer);  
}
```

Lệnh 'c': Ghi file CSV tương tự lệnh 'w', nhưng ghi vào file DATA.CSV. Định dạng CSV thường dùng dấu phẩy để ngăn cách dữ liệu, có thể mở bằng Exce

```
else if (cmd == 'v' || cmd == 'V') {  
    SD_Read_File("DATA.CSV");  
}
```

Lệnh 'v': Đọc file CSV Đọc và hiển thị nội dung file DATA.CSV

```
else {
    HAL_UART_Transmit(&huart1, (uint8_t*)" -> Lenh khong hop le!\r\n", 23, 100);
}
```

Lệnh không hợp lệ , lỗi nếu người dùng nhập ký tự không nằm trong danh sách quy định

+ Hàm SystemClock_Config

```
void SystemClock_Config(void)
{
    // ... (Khai báo struct)
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
    RCC_OscInitStruct.HSEState = RCC_HSE_ON;
    // ...
    RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9;
```

Cấu hình bộ dao động thạch anh ngoại (HSE) làm nguồn clock chính. Sử dụng bộ nhân tần số PLL (nhân 9) để đạt tốc độ CPU mong muốn (thường là 72MHz với thạch anh 8MHz)

```
RCC_ClkInitStruct.ClockType = ...;
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
// ...
```

Phân chia xung nhịp cho các đường bus AHB, APB1, APB2 để cấp cho các ngoại vi

+ Hàm PUTCHAR_PROTOTYPE (Retarget Printf)

```
PUTCHAR_PROTOTYPE
{
    HAL_UART_Transmit(&huart1, (uint8_t *)&ch, 1, 100);
    return ch;
}
```

Mặc định printf in ra màn hình máy tính. Hàm này định nghĩa lại hành vi đó: mỗi khi printf muốn in 1 ký tự (ch), nó sẽ gọi hàm này để gửi ký tự đó qua UART1 (HAL_UART_Transmit)

+Hàm Error_Handler

```
void Error_Handler(void)
{
    __disable_irq();
    while (1)
    {
    }
}
```

Nếu quá trình khởi tạo phần cứng (Clock, GPIO, SPI...) gặp lỗi nghiêm trọng, chương trình sẽ nhảy vào đây. Và hàm `__disable_irq()`: Ngắt toàn bộ ngắt hệ thống

- Giải thích code `sd_card.c`

+ Hàm `SD_Init_Mount` - Khởi tạo và Gắn thẻ

```
void SD_Init_Mount(void) {
    HAL_UART_Transmit(&huart1, (uint8_t*)"Dang chuan bi mount the nho (Doi 1-5s)...\r\n", 43, 1000);

    HAL_Delay(100); // Cho mot chut de nguon on dinh
```

Thông báo cho người dùng biết quá trình bắt đầu và chờ 100ms để điện áp ổn định.

```
fresult = f_mount(&fs, "/", 1);
```

`&fs`: Địa chỉ biến hệ thống tệp, `"/"`: Đường dẫn gốc, `1`: Tùy chọn "Mount ngay lập tức" Nếu là 0, nó sẽ hoãn việc mount cho đến khi có truy cập file đầu tiên.

```
if (fresult != FR_OK) {
    snprintf(sd_msg, sizeof(sd_msg), "LOI Mount: %d. Thu Format...\r\n", fresult);
    HAL_UART_Transmit(&huart1, (uint8_t*)sd_msg, strlen(sd_msg), 1000);
```

Nếu mount thất bại (`fresult` khác `FR_OK`), in mã lỗi ra màn hình

```
if (fresult == FR_NO_FILESYSTEM) {
    fresult = f_mkfs("", 1, 0); // Format
    if (fresult == FR_OK) {
        HAL_UART_Transmit(&huart1, (uint8_t*)"Format OK! Remounting...\r\n", 26, 1000);
        f_mount(NULL, "/", 1); // Unmount
        fresult = f_mount(&fs, "/", 1); // Remount
    }
}
}
```

Xử lý trường hợp đặc biệt: Thẻ chưa được định dạng (FR_NO_FILESYSTEM)

f_mkfs("", 1, 0): Lệnh Format thẻ nhớ sang định dạng FAT

Sau khi Format thành công, thực hiện Unmount (f_mount(NULL...)) và Mount lại từ đầu

```
if (fresult == FR_OK) {
    HAL_UART_Transmit(&huart1, (uint8_t*)" -> Mount THANH CONG!\r\n", 22, 1000);
} else {
    HAL_UART_Transmit(&huart1, (uint8_t*)" -> Mount THAT BAI!\r\n", 20, 1000);
}
}
```

Thông báo kết quả cuối cùng cho người dùng

+ Hàm SD_Write_File - Ghi dữ liệu vào file

```
void SD_Write_File(char *filename, char *data) {
    fresult = f_open(&fil, filename, FA_OPEN_ALWAYS | FA_WRITE);
```

Mở file gồm filename: Tên file (ví dụ "DATA.TXT"), FA_OPEN_ALWAYS: Nếu file chưa có thì tạo mới, nếu có rồi thì mở nó ra, FA_WRITE: Mở với quyền ghi

```
if (fresult == FR_OK) {
    f_lseek(&fil, f_size(&fil)); // Di chuyển xuống cuối file
```

Ghi nối tiếp gồm f_size(&fil): Lấy kích thước hiện tại của file và f_lseek: Di chuyển con trỏ ghi đến vị trí cuối cùng đó. Nếu không có dòng này, dữ liệu mới sẽ ghi đè lên dữ liệu cũ ở đầu file

```
f_puts(data, &fil);
f_puts("\n", &fil); // Xuống dòng
f_close(&fil);
HAL_UART_Transmit(&huart1, (uint8_t*)" -> Ghi file OK!\r\n", 17, 100);
```

f_puts: Ghi chuỗi data vào file. Ghi thêm ký tự xuống dòng \n để lần ghi sau dữ liệu nằm ở dòng mới. f_close: Đóng file để lưu thay đổi vật lý lên thẻ nhớ (rất quan trọng, nếu không đóng file dữ liệu sẽ bị mất)

+ Hàm SD_Read_File - Đọc file

```
void SD_Read_File(char *filename) {
    char read_buf[100];
    HAL_UART_Transmit(&huart1, (uint8_t*)"--- Noi dung file ---\r\n", 23, 100);

    fresult = f_open(&fil, filename, FA_READ);
```

Mở file với cờ FA_READ (chỉ đọc). read_buf là bộ đệm tạm để chứa từng dòng đọc được

```
if (fresult == FR_OK) {
    while (f_gets(read_buf, sizeof(read_buf), &fil)) {
        HAL_UART_Transmit(&huart1, (uint8_t*)read_buf, strlen(read_buf), 100);
    }
    f_close(&fil);
```

f_gets: Đọc một dòng từ file vào read_buf. Hàm này trả về NULL khi đọc đến cuối file (EOF), giúp vòng lặp while tự động dừng, HAL_UART_Transmit: Gửi dòng vừa đọc được lên máy tính

+ Hàm SD_Check_Capacity - Kiểm tra dung lượng

```
void SD_Check_Capacity(void) {
    fresult = f_getfree("", &fre_clust, &pfs);
```

Lấy thông tin về số cluster còn trống. &fre_clust: Biến sẽ nhận giá trị số cluster trống. &pfs: Con trỏ hệ thống tệp sẽ được cập nhật thông tin

```
if (fresult == FR_OK) {
    total = (uint32_t)((pfs->n_fatent - 2) * pfs->csize * 0.5);
    free_space = (uint32_t)(fre_clust * pfs->csize * 0.5);
```

Tính toán dung lượng ra đơn vị KB. pfs->csize: Số sector trên mỗi cluster. 0.5: Vì mỗi sector thường là 512 bytes = 0.5 KB. Công thức: Số cluster * Số sector/cluster * 0.5 KB = Dung lượng KB

```
snprintf(sd_msg, sizeof(sd_msg), "Tong: %lu KB, Trong: %lu KB\r\n", total, free_space);
HAL_UART_Transmit(&huart1, (uint8_t*)sd_msg, strlen(sd_msg), 1000);
```

Định dạng chuỗi kết quả và in ra màn hình

- Giải thích code fatfs_sd.c

+ Khai báo và Cấu hình

```
#include "stm32f1xx_hal.h"
#include "diskio.h"
#include "fatfs_sd.h"
#include "main.h"

extern SPI_HandleTypeDef hspi1;
#define HSPI_SDCARD &hspi1
```

Khai báo sử dụng hspi1 (SPI1) để giao tiếp, Định nghĩa các macro SD_CS_PORT và SD_CS_PIN để điều khiển chân Chip Select (CS). Chân này quyết định khi nào vi điều khiển nói chuyện với thẻ nhớ

```
extern volatile uint16_t Timer1, Timer2;
```

Đây là 2 biến đếm thời gian quan trọng. Chúng được khai báo extern vì chúng được giảm giá trị (trừ đi 1) mỗi mili-giây trong ngắt hệ thống (SysTick_Handler trong file stm32f1xx_it.c). Chúng dùng để tạo Timeout (thời gian chờ tối đa) để tránh chương trình bị treo nếu thẻ nhớ không phản hồi

+ Các hàm giao tiếp SPI cơ bản Đây là lớp vật lý, trực tiếp điều khiển tín hiệu điện
SELECT(): Kéo chân CS xuống mức thấp (0) để bắt đầu giao tiếp với thẻ

```
static void SELECT(void)
{
    HAL_GPIO_WritePin(SD_CS_PORT, SD_CS_PIN, GPIO_PIN_RESET);
    HAL_Delay(1);
}
```

DESELECT(): Kéo chân CS lên mức cao (1) để kết thúc giao tiếp

```
43  /* slave deselect */
44  static void DESELECT(void)
45  {
46      HAL_GPIO_WritePin(SD_CS_PORT, SD_CS_PIN, GPIO_PIN_SET);
47      HAL_Delay(1);
48  }
```

SPI_TxByte(uint8_t data): Gửi 1 byte dữ liệu qua SPI

```

50  /* SPI transmit a byte */
51  static void SPI_TxByte(uint8_t data)
52  {
53      while(!__HAL_SPI_GET_FLAG(HSPI_SDCARD, SPI_FLAG_TXE));
54      HAL_SPI_Transmit(HSPI_SDCARD, &data, 1, SPI_TIMEOUT);
55  }

```

SPI_RxByte(): Nhận 1 byte từ thẻ nhớ. Trong SPI, để nhận 1 byte, ta phải gửi đi 1 byte "giả" (dummy byte, thường là 0xFF)

```

56
57  /* SPI transmit buffer */
58  static void SPI_TxBuffer(uint8_t *buffer, uint16_t len)
59  {
60      while(!__HAL_SPI_GET_FLAG(HSPI_SDCARD, SPI_FLAG_TXE));
61      HAL_SPI_Transmit(HSPI_SDCARD, buffer, len, SPI_TIMEOUT);
62  }
63

```

+ Các hàm giao thức thẻ SD (SD Functions) là lớp xử lý logic của thẻ nhớ SD

SD_ReadyWait(): Chờ cho thẻ nhớ sẵn sàng. Thẻ nhớ sẽ gửi về 0xFF khi nó rảnh. Sử dụng Timer2 để timeout nếu chờ quá lâu

```

86  /* wait SD ready */
87  static uint8_t SD_ReadyWait(void)
88  {
89      uint8_t res;
90
91      /* timeout 500ms */
92      Timer2 = 100; // Giảm xuống 100 (tương đương 1 giây) để báo lỗi nhanh hơn
93
94      /* if SD goes ready, receives 0xFF */
95      do {
96          res = SPI_RxByte();
97      } while ((res != 0xFF) && Timer2);
98
99      return res;
100 }

```

SD_PowerOn(): Hàm này thực hiện quy trình "đánh thức" thẻ nhớ khi mới cấp điện. Gửi 80 xung clock (10 byte 0xFF) khi chân CS ở mức cao để thẻ nhớ ổn định điện áp. Gửi lệnh CMD0 (GO_IDLE_STATE) để đưa thẻ về trạng thái nghỉ (Idle)

```

101
102  /* power on */
103  static void SD_PowerOn(void)
104  {
105      uint8_t args[6];
106      uint32_t cnt = 0x1FFF;
107
108      /* transmit bytes to wake up */
109      DESELECT();
110      for(int i = 0; i < 10; i++)
111      {
112          SPI_TxByte(0xFF);
113      }
114

```

SD_SendCmd(BYTE cmd, uint32_t arg): Đây là hàm quan trọng nhất để gửi lệnh. Cấu trúc một lệnh SD gồm 6 bytes: Byte lệnh (0x40 | chỉ số lệnh), 4 bytes tham số (Argument), 1 byte CRC (kiểm tra lỗi). Hàm này chờ phản hồi từ thẻ (Response) và trả về kết quả

```

223  /* transmit command */
224  static BYTE SD_SendCmd(BYTE cmd, uint32_t arg)
225  {
226      uint8_t crc, res;
227
228      /* wait SD ready */
229      if (SD_ReadyWait() != 0xFF) return 0xFF;
230
231      /* transmit command */
232      SPI_TxByte(cmd); /* Command */
233      SPI_TxByte((uint8_t)(arg >> 24)); /* Argument[31..24] */
234      SPI_TxByte((uint8_t)(arg >> 16)); /* Argument[23..16] */
235      SPI_TxByte((uint8_t)(arg >> 8)); /* Argument[15..8] */
236      SPI_TxByte((uint8_t)arg); /* Argument[7..0] */
237
238      /* prepare CRC */
239      if(cmd == CMD0) crc = 0x95; /* CRC for CMD0(0) */
240      else if(cmd == CMD8) crc = 0x87; /* CRC for CMD8(0x1AA) */
241      else crc = 1;
242
243      /* transmit CRC */
244      SPI_TxByte(crc);
245
246      /* Skip a stuff byte when STOP_TRANSMISSION */
247      if (cmd == CMD12) SPI_RxByte();
248
249      /* receive response */
250      uint8_t n = 10;
251      do {
252          res = SPI_RxByte();
253      } while ((res & 0x80) && --n);
254
255      return res;
256  }

```


SD_RxDataBlock: Nhận một khối dữ liệu (thường là 512 bytes/sector). Nó chờ token bắt đầu (0xFE) rồi mới đọc dữ liệu

```
153 static bool SD_RxDataBlock(BYTE *buff, UINT len)
154 {
155     uint8_t token;
156
157     /* timeout 200ms */
158     Timer1 = 200;
159
160     /* loop until receive a response or timeout */
161     do {
162         token = SPI_RxByte();
163     } while((token == 0xFF) && Timer1);
164
165     /* invalid response */
166     if(token != 0xFE) return FALSE;
167
168     /* receive data */
169     do {
170         SPI_RxBytePtr(buff++);
171     } while(len--);
172
173     /* discard CRC */
174     SPI_RxByte();
175     SPI_RxByte();
176
177     return TRUE;
178 }
```

SD_TxDataBlock: Gửi một khối dữ liệu xuống thẻ để ghi

```
182 static bool SD_TxDataBlock(const uint8_t *buff, BYTE token)
183 {
184     uint8_t resp;
185     uint8_t i = 0;
186
187     /* wait SD ready */
188     if (SD_ReadyWait() != 0xFF) return FALSE;
189
190     /* transmit token */
191     SPI_TxByte(token);
192
193     /* if it's not STOP token, transmit data */
```

```

193     /* if it's not STOP token, transmit data */
194     if (token != 0xFD)
195     {
196         SPI_TxBuffer((uint8_t*)buff, 512);
197
198         /* discard CRC */
199         SPI_RxByte();
200         SPI_RxByte();
201
202         /* receive response */
203         while (i <= 64)
204         {
205             resp = SPI_RxByte();
206
207             /* transmit 0x05 accepted */
208             if ((resp & 0x1F) == 0x05) break;
209             i++;
210         }
211
212         /* recv buffer clear */
213         while (SPI_RxByte() == 0);
214     }
215

```

+ Các hàm giao diện DiskIO (User DiskIO Functions) là các hàm mà thư viện FatFs sẽ gọi trực tiếp. Chúng tuân theo chuẩn của diskio.h

SD_disk_initialize(BYTE drv) - Khởi tạo thẻ , hàm này thực hiện quy trình khởi tạo phức tạp để xác định loại thẻ (SDSC, SDHC, v.v.):CMD0: Reset thẻ,CMD8: Kiểm tra điện áp hoạt động (quan trọng cho thẻ SD v2.0),ACMD41: Lệnh khởi tạo chính thức. Nó được gửi lặp lại cho đến khi thẻ báo sẵn sàng,CMD58: Đọc thanh ghi OCR để xem thẻ là dung lượng chuẩn (SDSC) hay dung lượng cao (SDHC/SDXC). Biến CardType sẽ lưu loại thẻ này, ảnh hưởng đến cách tính địa chỉ bộ nhớ sau này

```

263     DSTATUS SD_disk_initialize(BYTE drv)
264     {
265         uint8_t n, type, ocr[4];

```

SD_disk_read(...) - Đọc Sector

```

DRESULT SD_disk_read(BYTE pdrv, BYTE* buff, DWORD sector, UINT count)

```

Đọc count sector (mỗi sector 512 bytes) bắt đầu từ địa chỉ sector vào bộ đệm buff. Nếu là thẻ thường (SDSC), địa chỉ phải nhân với 512 (sector *= 512). Thẻ SDHC tự hiểu theo block, Sử dụng CMD17 (đọc 1 block) hoặc CMD18 (đọc nhiều block)

SD_disk_write(...) - Ghi Sector

```
DRESULT SD_disk_write(BYTE pdrv, const BYTE* buff, DWORD sector, UINT count)
```

Ghi dữ liệu từ buff xuống thẻ nhớ. Sử dụng CMD24 (ghi 1 block) hoặc CMD25 (ghi nhiều block)

SD_disk_ioctl(...) - Điều khiển I/O

```
DRESULT SD_disk_ioctl(BYTE drv, BYTE ctrl, void *buff)
```

Cung cấp các thông tin phụ trợ cho FatFs. CTRL_SYNC: Chờ quá trình ghi hoàn tất. GET_SECTOR_COUNT: Trả về tổng số sector (để tính dung lượng thẻ). GET_SECTOR_SIZE: Trả về kích thước sector (luôn là 512 bytes với thẻ SD)

=> Tóm tắt luồng hoạt động ở fatfs_sd.c : gọi f_mount hoặc f_open ở main.c:

- + FatFs gọi disk_initialize (trong diskio.c, file này gọi SD_disk_initialize ở đây).
- + SD_disk_initialize dùng SD_SendCmd để gửi lệnh SPI khởi động thẻ.
- + Khi bạn gọi f_read, FatFs gọi SD_disk_read.
- + SD_disk_read gửi lệnh đọc sector qua SPI, nhận 512 bytes và đưa về cho bạn.

- Giải thích code spi.h

Chịu trách nhiệm cấu hình phần cứng giao tiếp SPI (Serial Peripheral Interface), để vi điều khiển STM32 gửi và nhận dữ liệu với thẻ nhớ SD

- + Khai báo và Biến toàn cục

```
#include "spi.h"

SPI_HandleTypeDef hspi1;
```

#include "spi.h": Gọi file header chứa các nguyên mẫu hàm và SPI_HandleTypeDef
hspi1: Khai báo biến cấu trúc hspi1. Đây là "thẻ căn cước" của bộ SPI1. Mọi hàm HAL liên quan đến SPI1 (như gửi, nhận) đều cần truyền biến này vào để biết đang điều khiển bộ SPI nào

+ Hàm MX_SPI1_Init - Cấu hình tham số SPI ,thiết lập các thông số hoạt động cho SPI (tốc độ, chế độ, cách gửi dữ liệu...)

```
void MX_SPI1_Init(void)
{
    hspi1.Instance = SPI1;
```

Chọn bộ SPI phần cứng số 1 (STM32F103 thường có SPI1, SPI2...). SPI1 thường có tốc độ cao nhất vì nằm trên bus APB2

```
hspi1.Init.Mode = SPI_MODE_MASTER;
```

Cấu hình STM32 làm Master .STM32 sẽ tạo ra xung nhịp để điều khiển thẻ nhớ

```
hspi1.Init.Direction = SPI_DIRECTION_2LINES;
```

Gửi dữ liệu theo gói 8-bit (1 byte). Đây là chuẩn của thẻ nhớ SD

```
hspi1.Init.CLKPolarity = SPI_POLARITY_LOW;
hspi1.Init.CLKPhase = SPI_PHASE_1EDGE;
```

Thiết lập SPI Mode 0 (CPOL=0, CPHA=0). POLARITY_LOW: Khi không truyền, dây Clock ở mức thấp (0V).PHASE_1EDGE: Dữ liệu được lấy mẫu ở cạnh lên (cạnh đầu tiên) của xung Clock

Lưu ý: Thẻ nhớ SD hỗ trợ Mode 0 và Mode 3. Cấu hình này là chuẩn

```
hspi1.Init.NSS = SPI_NSS_SOFT;
```

Quản lý chân CS (Chip Select) bằng Phần mềm (Software).Thay vì để bộ SPI tự động kéo chân CS, dùng lệnh HAL_GPIO_WritePin (trong file fatfs_sd.c) để chọn thẻ nhớ khi cần. Điều này rất quan trọng với driver thẻ nhớ vì quy trình khởi tạo cần thao tác chân CS rất linh hoạt

```
hspi1.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_128;
```

Bộ chia tần số để thiết lập tốc độ truyền (Baudrate). Giả sử Clock hệ thống là 72MHz (APB2 = 72MHz). Tốc độ SPI = 72MHz / 128 = **562.5 kHz**.

Tại sao chậm vậy? Khi mới khởi tạo (Initialization), thẻ nhớ SD yêu cầu tốc độ **dưới 400kHz**. Sau khi khởi tạo xong, ta có thể tăng tốc độ lên (ví dụ chia 4 hoặc 2) để đọc ghi nhanh hơn (như trong đoạn code bị comment ở sd_card.c)

```
hspi1.Init.FirstBit = SPI_FIRSTBIT_MSB;
```

Gửi bit trọng số cao nhất (MSB) đi trước. Đây là chuẩn giao tiếp của thẻ SD

```
hspi1.Init.TIMode = SPI_TIMODE_DISABLE;  
hspi1.Init.CRCCalculation = SPI_CRCCalculation_DISABLE;
```

Tắt chế độ TI (Texas Instruments) và tắt tính toán CRC phần cứng (vì driver fatfs_sd.c đã tự tính CRC mềm hoặc bỏ qua CRC)

```
if (HAL_SPI_Init(&hspi1) != HAL_OK)  
{  
    Error_Handler();  
}
```

Áp dụng các cấu hình trên. Nếu lỗi thì nhảy vào Error_Handler để treo chip

+ Hàm HAL_SPI_MspInit - Khởi tạo phần cứng cấp thấp

Hàm này được HAL_SPI_Init gọi tự động. Nó cấu hình Chân và Clock cho SPI

```
void HAL_SPI_MspInit(SPI_HandleTypeDef* spiHandle)  
{  
    GPIO_InitTypeDef GPIO_InitStruct = {0};  
    if(spiHandle->Instance==SPI1)  
    {  
        /* SPI1 clock enable */  
        __HAL_RCC_SPI1_CLK_ENABLE();  
        __HAL_RCC_GPIOA_CLK_ENABLE();  
    }
```

Cấp xung nhịp cho bộ SPI1 và port GPIOA (nơi chứa các chân SPI1). Nếu không có dòng này, SPI sẽ không chạy

```
/**SPI1 GPIO Configuration
PA5      -----> SPI1_SCK
PA6      -----> SPI1_MISO
PA7      -----> SPI1_MOSI
*/
GPIO_InitStruct.Pin = GPIO_PIN_5|GPIO_PIN_7;
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
```

Cấu hình chân **SCK (PA5)** và **MOSI (PA7)**. GPIO_MODE_AF_PP: Alternate Function Push-Pull. Nghĩa là chân này không dùng làm GPIO thường mà được bàn giao cho bộ SPI điều khiển xuất tín hiệu

```
GPIO_InitStruct.Pin = GPIO_PIN_6;
GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
```

Cấu hình chân **MISO (PA6)** , GPIO_MODE_INPUT: Chân này nhận dữ liệu từ thẻ nhớ về, nên phải là Input.

Lưu ý: Chân **CS (Chip Select)** không nằm ở đây vì nó được cấu hình là SPI_NSS_SOFT. Nó được khai báo là một chân GPIO Output bình thường (thường là PA4) trong file gpio.c

+ Hàm HAL_SPI_MspDeInit - Hủy khởi tạo

```
void HAL_SPI_MspDeInit(SPI_HandleTypeDef* spiHandle)
{
    if(spiHandle->Instance==SPI1)
    {
        __HAL_RCC_SPI1_CLK_DISABLE(); // Ngắt clock SPI1
        HAL_GPIO_DeInit(GPIOA, GPIO_PIN_5|GPIO_PIN_6|GPIO_PIN_7); // Đưa các chân về trạng thái mặc định
    }
}
```

Hàm này dùng để tắt SPI hoàn toàn, thường ít dùng trong ứng dụng này trừ khi bạn muốn tiết kiệm điện năng tối đa khi không dùng thẻ nhớ

=> Tóm tắt luồng dữ liệu:

- + fatfs_sd.c muốn gửi byte 0xFF.

- + Nó gọi HAL_SPI_Transmit(&hspi1, ...) (sử dụng biến hspi1 từ spi.c).

- + spi.c (thông qua HAL) điều khiển chân PA5 (SCK) tạo xung nhịp và đẩy bit ra chân PA7 (MOSI).

- + Đồng thời, nó thu bit từ chân PA6 (MISO) về.